

SOC MONITORING & INCIDENT DETECTION LAB

A Hands-on Blue Team / SOC Project

Prepared by:

Komal Azam

Degree:

BS Information Technology

Tools & Technologies:

Wazuh | Suricata | Windows | Ubuntu | VirtualBox | Nmap | MITRE ATT&CK

Project Type:

Cybersecurity / Blue Team / SOC

Year: 2026

TABLE OF CONTENTS

1. Introduction

- 1.1 Project Overview
- 1.2 Problem Statement
- 1.3 Project Purpose
- 1.4 Project Scope

2. Project Objectives

- 2.1 Primary Objectives
- 2.2 Learning Objectives

3. Lab Environment & Architecture

- 3.1 Lab Environment
- 3.2 Hardware & Virtualization
- 3.3 Network Architecture
- 3.4 Security Tools & Components
- 3.5 Overall Architecture

4. Lab Setup & Implementation

- 4.1 Day 1 — Environment Preparation
- 4.2 Day 2 — Ubuntu & Wazuh Manager Setup
- 4.3 Day 3 — Wazuh Dashboard & Log Collection
- 4.4 Day 4 — SSH Brute-Force Detection
- 4.5 Day 5 — Network Scanning Using Nmap
- 4.6 Day 6 — File Integrity Monitoring (FIM)
- 4.7 Day 7 — Suspicious User Creation
- 4.8 Day 8 — Suricata IDS Monitoring
- 4.9 Day 9 — MITRE ATT&CK Mapping & Incident Investigation
- 4.10 Day 10 — Documentation & Finalization

5. Security Monitoring & Incident Detection

- 5.1 Wazuh Security Monitoring
- 5.2 Security Alerts & Detection

6. Incident Investigation & MITRE ATT&CK

- 6.1 Incident Investigation Process
- 6.2 Incident Analysis
- 6.3 MITRE ATT&CK Mapping

7. Challenges & Troubleshooting

- 7.1 Challenges Faced
- 7.2 Troubleshooting

8. Results & Skills Demonstrated

8.1 Project Results & Findings

8.2 Skills Demonstrated

9. Conclusion

9.1 Project Summary

9.2 Key Learning Outcomes

9.3 Future Improvements

10. References

Appendices

Appendix A — Architecture Diagram

Appendix B — Selected Wazuh Alerts

Appendix C — Nmap Scan Evidence

Appendix D — Suricata Evidence

Appendix E — Windows Event Evidence

Appendix F — Incident Reports

1. Introduction

1.1 Project Overview

The **SOC Monitoring & Incident Detection Lab** is a hands-on cybersecurity project designed to simulate a basic Security Operations Center (SOC) environment. The project focuses on security monitoring, log collection, alert detection, incident investigation, and security event analysis using industry-relevant tools.

The laboratory environment consists of a Windows host and an Ubuntu virtual machine. The Ubuntu environment is used to host the Wazuh Manager and Wazuh Dashboard, while the Windows host is monitored through the Wazuh Agent. Suricata is also incorporated into the environment to support network-based security monitoring and intrusion detection.

The project involves generating controlled security activities within the laboratory environment and analyzing the resulting logs and alerts. The detected activities are investigated using available evidence and, where applicable, mapped to the MITRE ATT&CK framework.

1.2 Problem Statement

Modern computer systems continuously generate large amounts of security-related data, including authentication events, system changes, network activity, and file modifications. Manually reviewing these events can be difficult and time-consuming, especially when suspicious activities are mixed with normal system activity.

A Security Operations Center addresses this challenge by continuously monitoring systems, collecting security logs, detecting suspicious behavior, and investigating alerts. Security monitoring platforms such as SIEM solutions help security analysts centralize and analyze events from different sources.

This project addresses the need for practical SOC experience by creating a controlled laboratory environment in which security events can be generated, detected, investigated, and documented. The lab provides an opportunity to understand how security monitoring tools can be used to identify potentially suspicious activities and support incident investigation.

1.3 Project Purpose

The primary purpose of this project is to gain practical experience in **Blue Team and SOC operations** by building and operating a small-scale security monitoring laboratory.

The project aims to demonstrate the practical use of Wazuh for security monitoring, log analysis, and alert investigation, along with Suricata for network-based detection. It also provides practical experience in examining security events, identifying relevant evidence, understanding alert severity, and documenting investigation findings.

Through controlled security scenarios, the project demonstrates an end-to-end SOC workflow:

Detect → Validate → Investigate → Analyze → Respond → Document

The project also aims to develop practical skills that are relevant to entry-level SOC and cybersecurity roles, including SIEM monitoring, log analysis, endpoint monitoring, network security monitoring, incident documentation, and MITRE ATT&CK mapping.

1.4 Project Scope

The scope of this project covers the design, configuration, and operation of a controlled cybersecurity laboratory environment using VirtualBox, Windows, Ubuntu, Wazuh, and Suricata.

The project includes:

- Setting up an Ubuntu-based Wazuh monitoring environment.
- Configuring the Wazuh Manager and Wazuh Dashboard.
- Installing and configuring the Wazuh Agent on the Windows host.
- Monitoring endpoint and system activity.
- Configuring Suricata for network security monitoring.
- Generating controlled security events within the laboratory.
- Detecting suspicious activities through Wazuh and Suricata.
- Investigating alerts using logs and available evidence.
- Identifying source and target information where available.

- Analyzing alert severity and relevant event details.
- Mapping applicable incidents to MITRE ATT&CK techniques.
- Preparing structured incident reports.
- Documenting findings, evidence, responses, and recommendations.

The project is conducted in an isolated and controlled laboratory environment for educational and defensive cybersecurity purposes. The testing activities are limited to systems within the laboratory environment.

2. Project Objectives

The main objective of the **SOC Monitoring & Incident Detection Lab** is to develop practical experience in security monitoring, alert detection, incident investigation, and defensive cybersecurity operations.

The specific objectives of this project are:

1. **Build a controlled SOC laboratory environment** using VirtualBox, Windows, and Ubuntu.
2. **Deploy and configure Wazuh** as the central security monitoring and SIEM platform.
3. **Configure the Wazuh Agent** on the Windows host to collect and monitor endpoint security events.
4. **Implement Suricata** for network-based intrusion detection and security monitoring.
5. **Generate controlled security events** within the laboratory environment for detection and analysis.
6. **Detect suspicious activities** using Wazuh and Suricata alerts.
7. **Analyze security logs and alerts** to identify relevant information such as timestamps, source IP addresses, affected systems, users, rule IDs, and severity levels.
8. **Investigate security incidents** using a structured SOC workflow and supporting evidence.
9. **Map detected activities to the MITRE ATT&CK framework** where an applicable technique can be identified.

10. **Prepare professional incident reports** containing detection details, investigation findings, evidence, response actions, and recommendations.
11. **Develop practical Blue Team skills** in SIEM monitoring, log analysis, endpoint monitoring, network security monitoring, and incident documentation.
12. **Demonstrate an end-to-end SOC investigation process** from security event detection to analysis, response, and documentation.

3. Lab Environment & Architecture

3.1 Lab Environment

The SOC Monitoring & Incident Detection Lab was developed using a combination of a Windows host machine and an Ubuntu virtual machine running through VirtualBox. The environment was designed to provide a controlled platform for security monitoring, log collection, alert generation, and incident investigation.

The Windows host functions as a monitored endpoint through the Wazuh Agent, while the Ubuntu virtual machine provides the central security monitoring infrastructure. Wazuh Manager and Wazuh Dashboard are deployed on the Ubuntu environment to collect, process, and visualize security events.

Suricata is also used as a network intrusion detection component to monitor network activity and identify potentially suspicious traffic.

3.2 Hardware & Virtualization

The laboratory was implemented on a Windows 11 Pro host system using Oracle VirtualBox for virtualization.

The main components of the environment are:

Component	Role
Windows 11 Pro	Host and monitored endpoint
Ubuntu VM	Central SOC/SIEM environment
VirtualBox	Virtualization platform
Wazuh Agent	Endpoint monitoring
Wazuh Manager	Security event processing and monitoring
Wazuh Dashboard	Security alert visualization and investigation
Suricata	Network intrusion detection

The virtualized environment allows security monitoring and testing activities to be performed in a controlled laboratory without relying on external production systems.

3.3 Network Architecture

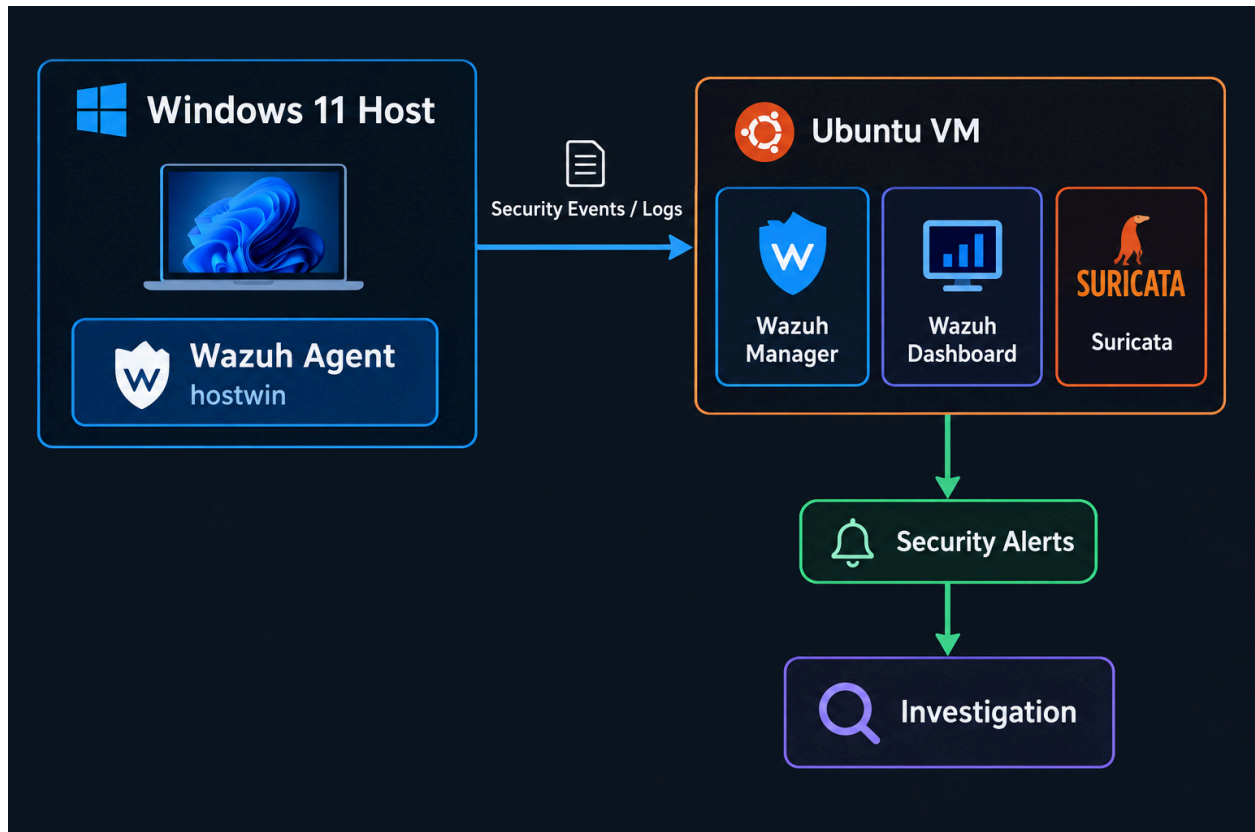
The Windows host and Ubuntu virtual machine communicate through the configured laboratory network.

The Ubuntu Wazuh server uses the following lab IP address:

Ubuntu Wazuh Server: 192.168.56.110

The Windows host is configured with the Wazuh Agent and communicates with the Wazuh Manager using the Ubuntu server's IP address.

The basic communication flow is:



This architecture provides a centralized location for collecting and analyzing security events generated within the laboratory environment.

3.4 Security Tools & Components

Wazuh

Wazuh is used as the primary security monitoring and SIEM platform. It provides capabilities for collecting security events, generating alerts, monitoring endpoints, and supporting security investigations.

Wazuh Agent

The Wazuh Agent is installed on the Windows host to collect relevant endpoint security information and communicate it to the Wazuh Manager.

The configured agent information includes:

- **Agent Name:** hostwin

- **Agent ID:** 002
- **Manager IP:** 192.168.56.110

Wazuh Dashboard

The Wazuh Dashboard provides a graphical interface for viewing security alerts, monitoring agents, searching events, and investigating detected activity.

Suricata

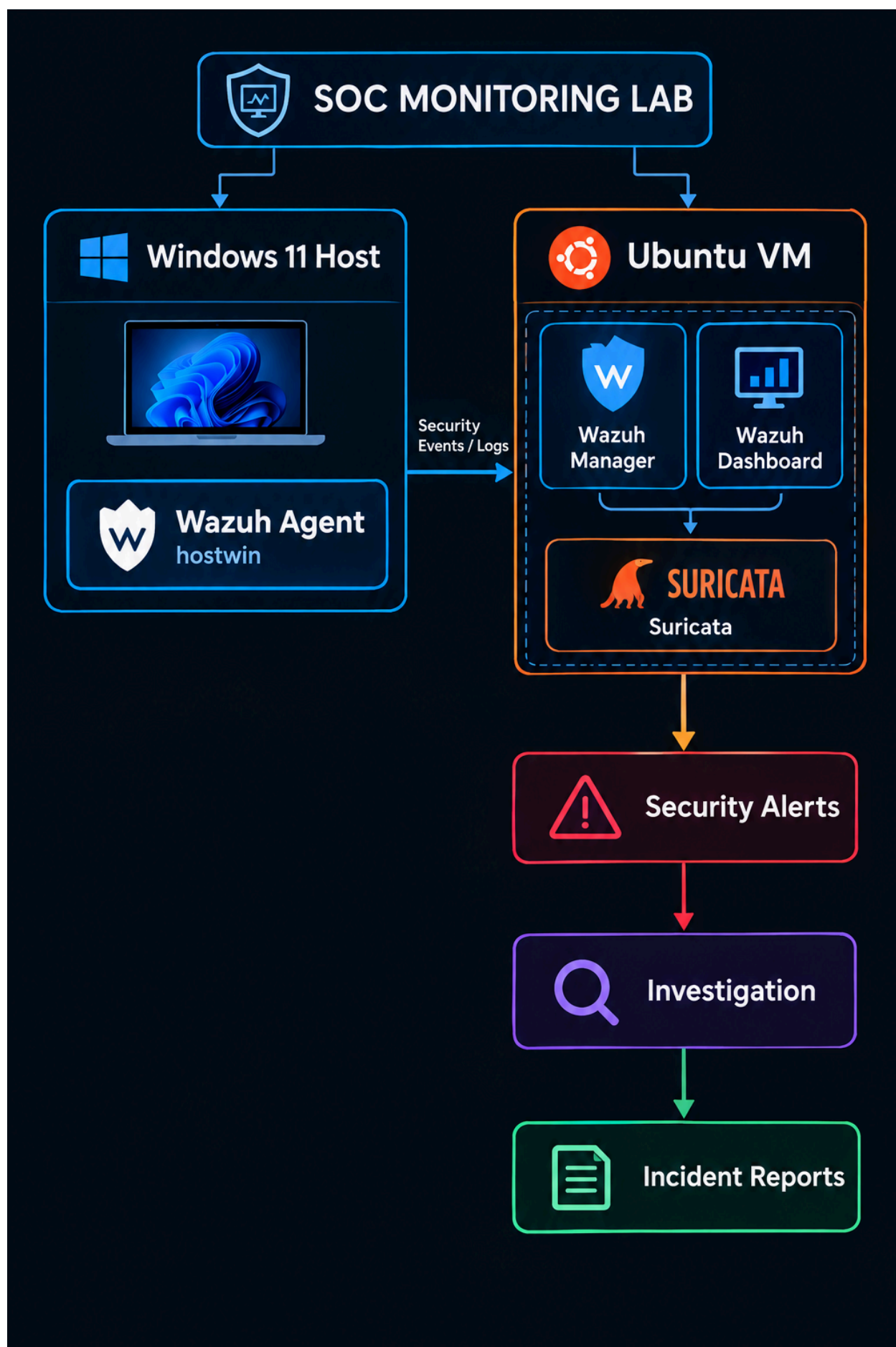
Suricata is used as a network intrusion detection component. It analyzes network traffic and can generate security alerts when traffic matches configured detection rules.

VirtualBox

VirtualBox is used to create and manage the Ubuntu virtual machine that hosts the SOC monitoring infrastructure.

3.5 Overall Architecture

The overall laboratory architecture can be summarized as follows:



The architecture was designed to support the complete security monitoring lifecycle, from event collection and detection to investigation and incident documentation.

4.1 Day 1 — Environment Preparation

The first stage of the SOC Monitoring & Incident Detection Lab focused on preparing the laboratory environment and establishing the virtualized infrastructure required for the project.

The Windows 11 Pro system was used as the host machine, while Oracle VirtualBox was used to create and manage the Ubuntu virtual machine. The Ubuntu VM was prepared to serve as the central environment for the security monitoring infrastructure.

4.1.1 Host System Preparation

The host system was prepared with the required virtualization software and resources for running the Ubuntu virtual machine.

The following components were used:

- **Host Operating System:** Windows 11 Pro
- **Virtualization Platform:** Oracle VirtualBox
- **Virtual Machine:** Ubuntu
- **Virtual Network:** Private laboratory network

4.1.2 Ubuntu Virtual Machine

An Ubuntu virtual machine was created in VirtualBox as part of the laboratory environment. The VM was prepared for the installation and configuration of the security monitoring tools used in later stages of the project.

The Ubuntu environment was assigned the private laboratory IP address:

Ubuntu IP Address: 192.168.56.110

This address was later used for communication between the Windows Wazuh Agent and the Wazuh Manager.

4.1.3 Network Configuration

The Windows host and Ubuntu virtual machine were configured to communicate through the private laboratory network.

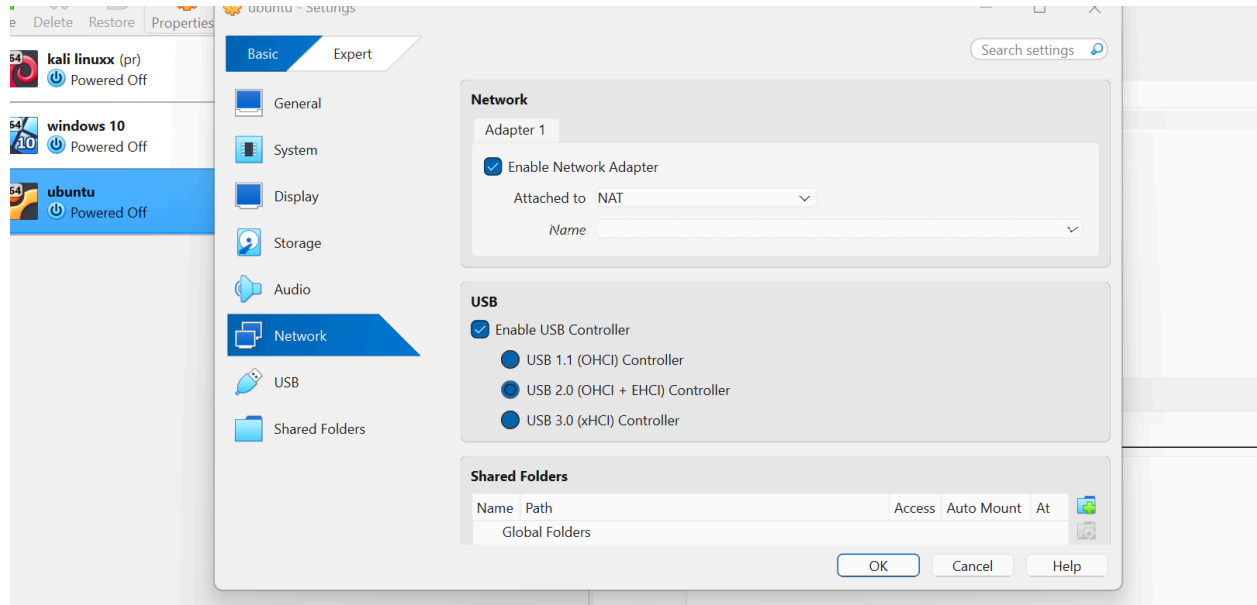
Basic connectivity was verified between the Windows host and Ubuntu VM to ensure that the systems could communicate before proceeding with the installation of the monitoring components.

4.1.4 Initial Environment Verification

After completing the initial virtualization and network configuration, the Ubuntu virtual machine was started and its network connectivity was verified.

The successful preparation of the environment provided the foundation for the subsequent installation and configuration of Wazuh Manager, Wazuh Dashboard, Wazuh Agent, and Suricata.

Note: No security testing or attack simulation was performed during this stage. Day 1 was limited to environment preparation and configuration.



Windows to Ubuntu connectivity/ping verification

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\system32> ping 192.168.56.110

Pinging 192.168.56.110 with 32 bytes of data:
Reply from 192.168.56.110: bytes=32 time<1ms TTL=64
Reply from 192.168.56.110: bytes=32 time<1ms TTL=64
Reply from 192.168.56.110: bytes=32 time<1ms TTL=64
Reply from 192.168.56.110: bytes=32 time<1ms TTL=64

Ping statistics for 192.168.56.110:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 0ms, Maximum = 0ms, Average = 0ms
PS C:\WINDOWS\system32> ^C
PS C:\WINDOWS\system32> _
```

4.2 Day 2 — Ubuntu & Wazuh Manager Setup

After preparing the laboratory environment, the second stage focused on configuring the Ubuntu virtual machine as the central security monitoring server.

Wazuh was selected as the primary security monitoring and SIEM platform for the laboratory. The Wazuh Manager was installed on the Ubuntu virtual machine to receive, process, and analyze security events generated by monitored systems.

4.2.1 Wazuh Manager Installation

The Ubuntu virtual machine was prepared for the installation of Wazuh. The required Wazuh components were installed and configured within the Ubuntu environment.

The Wazuh Manager was configured to operate on the Ubuntu server with the following laboratory network address:

Wazuh Manager IP: 192.168.56.110

The manager serves as the central component responsible for receiving security data from configured agents and processing the collected events.

4.2.2 Wazuh Service Verification

After installation, the Wazuh services were checked to verify that the monitoring infrastructure was functioning correctly.

The service status was verified using the appropriate Linux service-management commands. This step was performed to confirm that the Wazuh Manager was running successfully before proceeding with dashboard and agent configuration.

4.2.3 Wazuh Configuration

The Wazuh Manager configuration was reviewed and adjusted where required for the laboratory environment.

The configuration process established the Ubuntu VM as the central monitoring server to which the Windows endpoint would later connect through the Wazuh Agent.

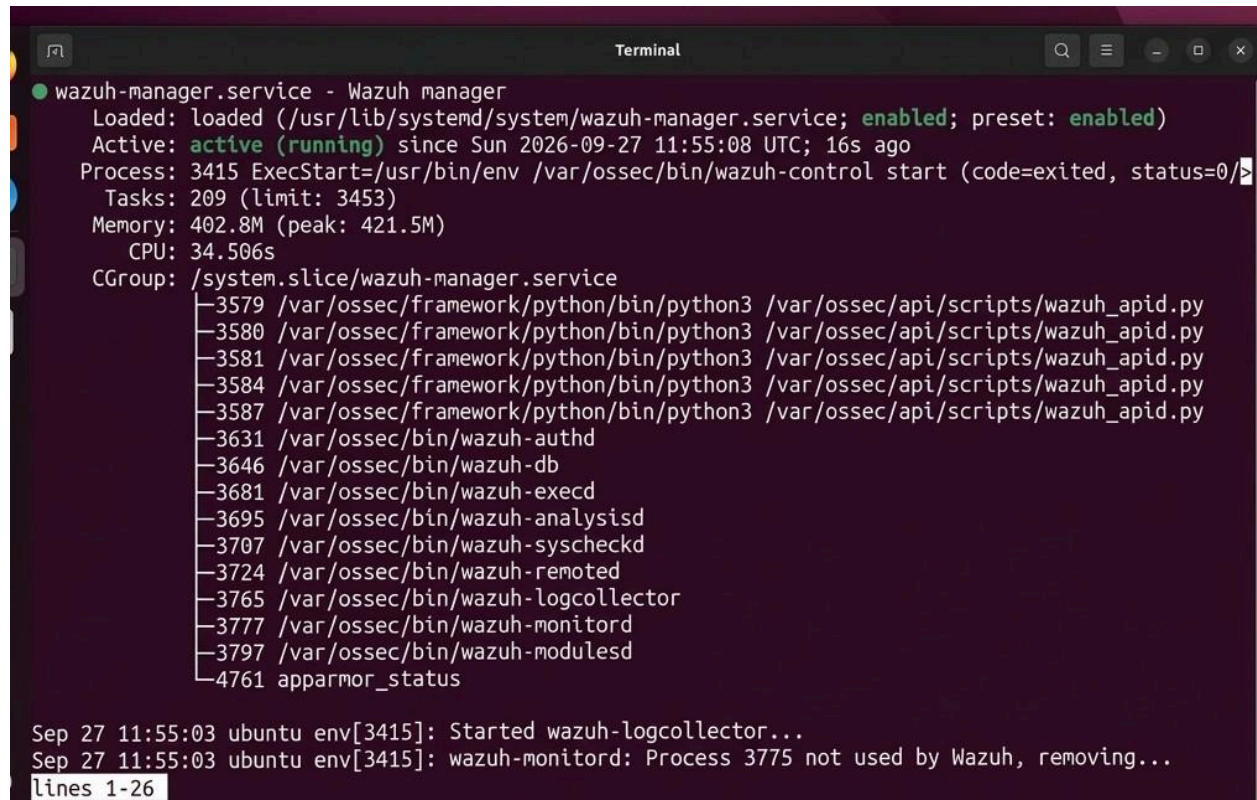
4.2.4 Initial Verification

The successful operation of the Wazuh Manager was verified before continuing to the next stage of the project.

At the end of Day 2, the Ubuntu VM was functioning as the central Wazuh monitoring server and was ready for the configuration of the Wazuh Dashboard and endpoint monitoring.

Day 2 Outcome:

The Wazuh Manager was successfully installed and prepared on the Ubuntu virtual machine, establishing the central security monitoring component of the laboratory.

A terminal window titled "Terminal" with a dark background. It displays the status of the "wazuh-manager.service" and a list of its active processes. The status shows it is loaded, active, and running. The process list includes several instances of wazuh_apid.py, wazuh-authd, wazuh-db, wazuh-execd, wazuh-analysisd, wazuh-syscheckd, wazuh-remoted, wazuh-logcollector, wazuh-monitor, wazuh-modulesd, and apparmor_status. At the bottom, there are two log messages from the system journal.

```
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-09-27 11:55:08 UTC; 16s ago
 Process: 3415 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/>>
    Tasks: 209 (limit: 3453)
   Memory: 402.8M (peak: 421.5M)
      CPU: 34.506s
   CGroup: /system.slice/wazuh-manager.service
           └─3579 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             └─3580 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
               └─3581 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                 └─3584 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                   └─3587 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
                     └─3631 /var/ossec/bin/wazuh-authd
                       └─3646 /var/ossec/bin/wazuh-db
                         └─3681 /var/ossec/bin/wazuh-execd
                           └─3695 /var/ossec/bin/wazuh-analysisd
                             └─3707 /var/ossec/bin/wazuh-syscheckd
                               └─3724 /var/ossec/bin/wazuh-remoted
                                 └─3765 /var/ossec/bin/wazuh-logcollector
                                   └─3777 /var/ossec/bin/wazuh-monitor
                                     └─3797 /var/ossec/bin/wazuh-modulesd
                                       └─4761 apparmor_status

Sep 27 11:55:03 ubuntu env[3415]: Started wazuh-logcollector...
Sep 27 11:55:03 ubuntu env[3415]: wazuh-monitor: Process 3775 not used by Wazuh, removing...
lines 1-26
```

4.3 Day 3 — Wazuh Dashboard & Log Collection

After successfully configuring the Wazuh Manager, the third stage of the project focused on accessing the Wazuh Dashboard and establishing the monitoring environment for security events.

The Wazuh Dashboard provides a graphical interface for monitoring connected agents, viewing security alerts, searching collected events, and supporting incident investigation.

4.3.1 Wazuh Dashboard Access

The Wazuh Dashboard was accessed from the Windows host through a web browser using the network address of the Ubuntu Wazuh server.

The dashboard provided access to the security monitoring interface and allowed the status of the configured monitoring environment to be reviewed.

4.3.2 Agent Monitoring

The Wazuh environment was prepared to monitor the Windows host through the Wazuh Agent.

The configured Windows agent was identified in the Wazuh environment with the following details:

- **Agent Name:** hostwin
- **Agent ID:** 002
- **Manager IP:** 192.168.56.110

The agent connection allowed security-related information from the Windows endpoint to be collected and processed by the Wazuh Manager.

4.3.3 Log Collection and Monitoring

After establishing the monitoring environment, collected security events were made available through the Wazuh Dashboard.

The dashboard was used to review security events and alerts generated from the monitored environment. Relevant event information could include timestamps, rule IDs, alert levels, source information, affected systems, and event descriptions.

This provided the foundation for investigating specific security scenarios in the later stages of the project.

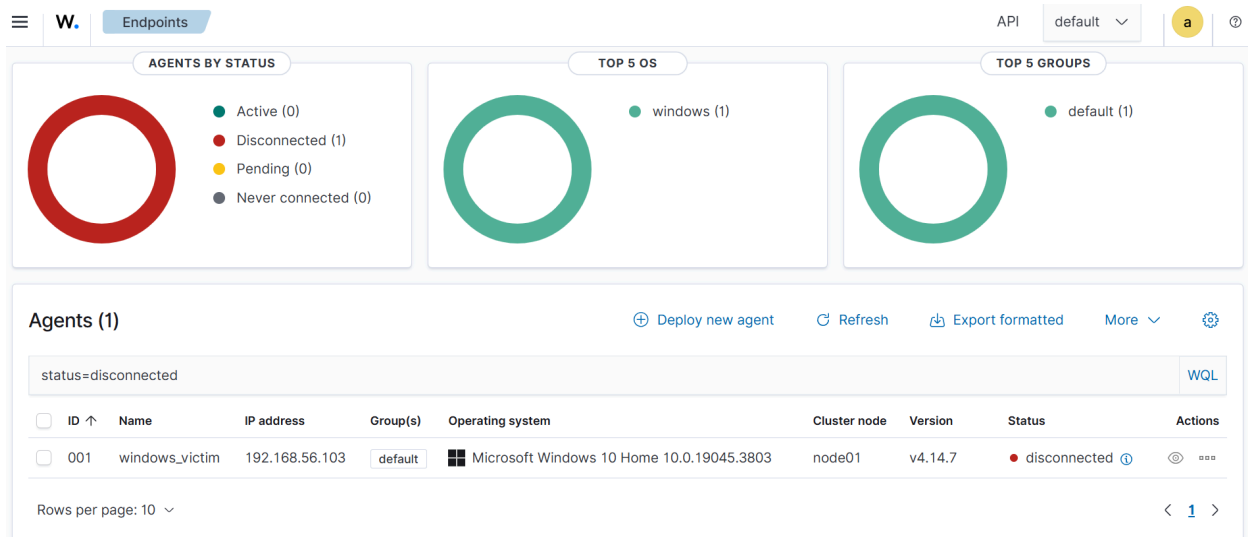
4.3.4 Initial Monitoring Verification

The Wazuh Dashboard and agent status were reviewed to verify that the monitoring environment was operational before beginning controlled security testing.

At the end of Day 3, the laboratory had a functioning Wazuh monitoring environment consisting of the Wazuh Manager, Wazuh Dashboard, and Windows endpoint agent.

Day 3 Outcome:

The Wazuh Dashboard was successfully accessed and the Windows endpoint was prepared for centralized security monitoring and log analysis.





W.

Endpoints

API

default

a



TOP 5 GROUPS



default (2)

Agents (2)

[Deploy new agent](#) [Refresh](#) [Export formatted](#) [More](#) [Settings](#)

status=active									WQL
☐	ID ↑	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
☐	001	windows_victim	192.168.56.103	default	Microsoft Windows 10 Home 10.0.19045.3803	node01	v4.14.7	active ⓘ	
☐	002	hostwin	192.168.56.1	default	Microsoft Windows 11 Pro 10.0.26200.9445	node01	v4.14.5	active ⓘ	

4.4 Day 4 — Controlled SSH Brute-Force Simulation and Detection

The fourth stage of the project focused on detecting and investigating a controlled SSH brute-force activity within the laboratory environment.

The purpose of this scenario was to understand how repeated failed SSH authentication attempts can be detected by Wazuh and how the resulting alerts can be investigated using security event information.

4.4.1 Test Scenario

Controlled SSH authentication attempts were performed from the Windows host against the Ubuntu server within the laboratory environment.

The testing generated multiple failed authentication events for the SSH service. These events were collected and processed by the Wazuh Manager.

The activity was performed only within the controlled laboratory environment for defensive security monitoring and investigation purposes.

4.4.2 Detection

Wazuh successfully detected the repeated SSH authentication failures and generated security alerts.

The observed events included:

- **Rule ID:** 5712
- **MITRE ATT&CK Technique:** T1110
- **Technique:** Brute Force
- **Authentication Failure Rule:** 5760
- **Alert Level:** Level 5 for the observed SSH authentication failure
- **Target User:** testuser

The alerts provided evidence that repeated authentication failures were being detected by the Wazuh monitoring system.

4.4.3 Investigation

The generated alerts were reviewed through the Wazuh monitoring interface to understand the characteristics of the detected activity.

The investigation focused on:

- Timestamp of the authentication attempts
- Source IP address
- Target system
- Target username
- SSH authentication status
- Wazuh rule ID
- Alert severity
- MITRE ATT&CK mapping

The source information associated with the authentication attempts was also reviewed as part of the investigation.

4.4.4 Analysis

The repeated failed authentication attempts indicated activity consistent with an SSH brute-force pattern.

The Wazuh alerts provided useful information for validating the activity and understanding how authentication-related security events can be detected through centralized monitoring.

The detection also demonstrated the value of correlating multiple authentication failures rather than examining individual failed login events in isolation.

4.4.5 MITRE ATT&CK Mapping

The detected activity was mapped by Wazuh to:

Field	Details
Tactic	Credential Access
Technique	T1110 — Brute Force
Activity	Repeated SSH authentication failures

4.4.6 Response

As this was a controlled laboratory exercise, no production system was affected. The primary response activity was to validate the alert, investigate the available event information, and document the findings.

For a real-world environment, repeated SSH authentication failures would require additional investigation and appropriate defensive measures based on the organization's security policies.

4.4.7 Evidence

The following evidence was collected during the investigation:

- Wazuh alert showing Rule ID **5712**
- SSH authentication failure event associated with Rule ID **5760**
- Target username **testuser**
- Source information recorded in the alert
- MITRE ATT&CK mapping to **T1110**

Day 4 Outcome:

The laboratory successfully demonstrated the detection and investigation of controlled SSH brute-force activity using Wazuh. The scenario provided practical experience in alert analysis, authentication-event investigation, severity assessment, and MITRE ATT&CK mapping.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\System32> Test-NetConnection 192.168.56.110 -Port 22

ComputerName      : 192.168.56.110
RemoteAddress     : 192.168.56.110
RemotePort        : 22
InterfaceAlias    : Ethernet 2
SourceAddress     : 192.168.56.1
TcpTestSucceeded  : True

PS C:\WINDOWS\System32> ssh testuser@192.168.56.110
The authenticity of host '192.168.56.110 (192.168.56.110)' can't be established.
ED25519 key fingerprint is SHA256:J8ootBly/KZ+kR/XsnnhkchAEh0haoz522bI4f5G87s.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.56.110' (ED25519) to the list of known hosts.
testuser@192.168.56.110's password:
Permission denied, please try again.
testuser@192.168.56.110's password:
Permission denied, please try again.
testuser@192.168.56.110's password:
testuser@192.168.56.110: Permission denied (publickey,password).
PS C:\WINDOWS\System32>
```

W. Threat Hunting

API default a

Sep 16, 2026 @ 11:46:48.5...	ubuntu	Successful login to root executed.	5	5712
Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Login session closed.	3	5502
Sep 16, 2026 @ 11:46:08.5...	ubuntu	syslog: User missed the password more than one time	10	2502
Sep 16, 2026 @ 11:46:08.5...	ubuntu	sshd: Attempt to login using a non-existent user	5	5710
Sep 16, 2026 @ 11:46:07.1...	ubuntu	syslog: User missed the password more than one time	10	2502
Sep 16, 2026 @ 11:46:07.1...	ubuntu	sshd: Attempt to login using a non-existent user	5	5710
Sep 16, 2026 @ 11:46:06.4...	ubuntu	sshd: brute force trying to get access to the system. Non existent user.	10	5712
Sep 16, 2026 @ 11:46:05.1...	ubuntu	sshd: Attempt to login using a non-existent user	5	5710
Sep 16, 2026 @ 11:46:02.4...	ubuntu	sshd: Attempt to login using a non-existent user	5	5710
Sep 16, 2026 @ 11:46:01.1...	ubuntu	sshd: Attempt to login using a non-existent user	5	5710
Sep 16, 2026 @ 11:45:56.4...	ubuntu	sshd: Attempt to login using a non-existent user	5	5710
Sep 16, 2026 @ 11:45:56.4...	ubuntu	sshd: Attempt to login using a non-existent user	5	5710
Sep 16, 2026 @ 11:45:54.4...	ubuntu	PAM: User login failed.	5	5503
Sep 16, 2026 @ 11:45:54.4...	ubuntu	PAM: User login failed.	5	5503
Sep 16, 2026 @ 11:45:52.4...	ubuntu	sshd: Attempt to login using a non-existent user	5	5710

W. Threat Hunting

API default a

Sep 16, 2026 @ 11:48:10.5...	ubuntu	PAM: Log
Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Log
Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Log
Sep 16, 2026 @ 11:46:48.5...	ubuntu	Successful
Sep 16, 2026 @ 11:46:48.5...	ubuntu	Successful
Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Log
Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Log
Sep 16, 2026 @ 11:46:08.5...	ubuntu	syslog: Us
Sep 16, 2026 @ 11:46:08.5...	ubuntu	sshd: Atte
Sep 16, 2026 @ 11:46:07.1...	ubuntu	syslog: Us
Sep 16, 2026 @ 11:46:07.1...	ubuntu	sshd: Atte
Sep 16, 2026 @ 11:46:06.4...	ubuntu	sshd: brut
Sep 16, 2026 @ 11:46:05.1...	ubuntu	sshd: Atte
Sep 16, 2026 @ 11:46:02.4...	ubuntu	sshd: Atte

Document Details

View surrounding documents View single document

Table JSON

t _index	wazuh-alerts-4.x-2026.09.16
t agent.id	000
t agent.name	ubuntu
t data.srcip	192.168.56.1
t data.srcuser	testuser
t decoder.name	sshd
t decoder.parent	sshd
t full_log	2026-09-16T06:46:04.890245+00:00 ubuntu sshd[15490]: Failed password for invalid user testuser from 192.168.56.1 port 65374 ssh2
t id	1789541165.976246
t input.type	log
t location	/var/log/auth.log
t manager.name	ubuntu
t predecoder.program.name	sshd

Rows per page: 15

W.

Threat Hunting

APIdefaulta

	Sep 16, 2026 @ 11:48:10.5...	ubuntu	PAM: Log
	Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Log
	Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Log
	Sep 16, 2026 @ 11:46:48.5...	ubuntu	Successfu
	Sep 16, 2026 @ 11:46:48.5...	ubuntu	Successfu
	Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Log
	Sep 16, 2026 @ 11:46:48.5...	ubuntu	PAM: Log
	Sep 16, 2026 @ 11:46:08.5...	ubuntu	syslog: Us
	Sep 16, 2026 @ 11:46:08.5...	ubuntu	sshd: Att
	Sep 16, 2026 @ 11:46:07.1...	ubuntu	syslog: Us
	Sep 16, 2026 @ 11:46:07.1...	ubuntu	sshd: Att
	Sep 16, 2026 @ 11:46:06.4...	ubuntu	sshd: brul
	Sep 16, 2026 @ 11:46:05.1...	ubuntu	sshd: Att
	Sep 16, 2026 @ 11:46:02.4...	ubuntu	sshd: Att

Rows per page: 15

Document Details

View surrounding documents

View single document

# rule.firedtimes	1
# rule.frequency	8
t rule.gdpr	IV_35.7.d, IV_32.2
t rule.groups	syslog, sshd, authentication_failures
t rule.hipaa	164.312.b
t rule.id	5712
# rule.level	10
rule.mail	false
t rule.mitre.id	T1110
t rule.mitre.tactic	Credential Access
t rule.mitre.technique	Brute Force
t rule.nist_800_53	SI.4, AU.14, AC.7
t rule.pci_dss	11.4, 10.2.4, 10.2.5
t rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
timestamp	Sep 16, 2026 @ 11:46:06.498

4.5 Day 5 — Network Scanning Using Nmap

The fifth stage of the project focused on network reconnaissance and service discovery using Nmap. The objective was to identify accessible services running on the Ubuntu server within the controlled laboratory environment.

4.5.1 Test Environment

A Kali Linux environment was used to perform the network scan against the Ubuntu server.

The Ubuntu server used in the laboratory had the following IP address:

Target IP: 192.168.56.110

The scan was conducted within the controlled laboratory network for educational and defensive security monitoring purposes.

4.5.2 Nmap Scanning

Nmap was used from the Kali Linux environment to perform a controlled TCP SYN scan against the Ubuntu server.

The command used for the scan was:

```
nmap -sS -T3 192.168.56.110
```

The `-sS` option performs a TCP SYN scan, while `-T3` uses a moderate timing template. The target address **192.168.56.110** represents the Ubuntu server within the laboratory network.

The scan results identified the following open ports:

Port	Protocol	Status
22	TCP	Open
443	TCP	Open

Port 22 indicated that an SSH service was accessible on the Ubuntu server, while port 443 indicated an accessible HTTPS-related service.

4.5.3 Investigation

The Nmap results were reviewed to determine which network services were accessible on the target system.

The investigation focused on:

- Source system: Kali Linux
- Target system: Ubuntu VM
- Target IP: 192 . 168 . 56 . 110
- Open TCP ports
- Identified services
- Potential network exposure

The scan provided a basic view of the network services exposed by the Ubuntu server within the laboratory environment.

4.5.4 Analysis

Network scanning is an important reconnaissance activity because it can help identify accessible services and provide information about a system's network exposure.

In this laboratory scenario, Nmap successfully identified TCP ports 22 and 443 as open on the Ubuntu server. The results demonstrated how a security analyst can use network scanning information to understand the services available on a monitored system.

4.5.5 Evidence

The primary evidence collected during this activity was the Nmap scan output showing the target IP address and identified open ports.

Evidence included:

- Nmap scan results
- Target IP address: 192 . 168 . 56 . 110
- Open port 22/TCP
- Open port 443/TCP
- Service information reported by Nmap

4.5.6 Response

Since the scan was performed against the project's own laboratory environment, no external or production system was affected.

In a real-world SOC environment, identified exposed services would be reviewed to determine whether they are authorized, required, and appropriately secured.

Day 5 Outcome:

The laboratory demonstrated the use of Nmap for network reconnaissance and service discovery. The activity provided practical experience in identifying accessible services and analyzing the network exposure of a monitored system.

```
Session Actions Edit View Help
64 bytes from 192.168.56.110: icmp_seq=9 ttl=64 time=0.251 ms
64 bytes from 192.168.56.110: icmp_seq=10 ttl=64 time=0.303 ms
64 bytes from 192.168.56.110: icmp_seq=11 ttl=64 time=0.432 ms
64 bytes from 192.168.56.110: icmp_seq=12 ttl=64 time=0.692 ms
64 bytes from 192.168.56.110: icmp_seq=13 ttl=64 time=0.934 ms
64 bytes from 192.168.56.110: icmp_seq=14 ttl=64 time=1.83 ms
64 bytes from 192.168.56.110: icmp_seq=15 ttl=64 time=0.598 ms
64 bytes from 192.168.56.110: icmp_seq=16 ttl=64 time=0.362 ms
64 bytes from 192.168.56.110: icmp_seq=17 ttl=64 time=0.631 ms
^C
--- 192.168.56.110 ping statistics ---
17 packets transmitted, 17 received, 0% packet loss, time 16314ms
rtt min/avg/max/mdev = 0.251/0.821/1.828/0.467 ms

(alpha@kali)~$
(alpha@kali)~$ sudo nmap -sS -T3 192.168.56.110
[sudo] password for alpha:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-17 13:36 IST
mass_dns: warning: Unable to determine any DNS servers. Reverse DNS is disabled. Try using --system-dns or specify valid servers with --dns-servers
Nmap scan report for 192.168.56.110
Host is up (0.00017s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
443/tcp   open  https
MAC Address: 08:00:27:66:EA:44 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.32 seconds
```

4.6 Day 6 — File Integrity Monitoring (FIM)

File Integrity Monitoring (FIM) was tested on the Windows host to observe whether changes to monitored files could be detected by Wazuh. A test file named **fim-test-2.txt** was created using PowerShell on the Windows host.

The file activity was monitored through the Wazuh Agent installed on the Windows system. After creating the test file, the Wazuh Dashboard was checked for the corresponding security event and file integrity alert.

Testing Activity:

- Operating System: Windows 11 Pro
- Monitoring Agent: hostwin
- Agent ID: 002
- Test File: fim-test-2.txt
- Method Used: PowerShell
- Monitoring Tool: Wazuh File Integrity Monitoring (FIM)

Investigation:

The PowerShell activity was used as the initial evidence showing the creation of the test file. The resulting Wazuh event was then reviewed to identify the file involved, event details, timestamp, rule information, and other available evidence.

The investigation demonstrated how FIM can help security analysts monitor file-system changes on an endpoint and investigate unexpected modifications.

Evidence:

- PowerShell screenshot showing creation of **fim-test-2.txt**
- Wazuh Dashboard screenshot showing the related FIM alert.

Outcome:

The FIM test demonstrated the process of generating a controlled file-system change on a Windows endpoint and reviewing the resulting security event through centralized Wazuh monitoring.

```
Mode                LastWriteTime         Length Name
----                -
-a-----          19/09/2026    21:50             0 fim-test-2.txt

PS C:\Users\user> Get-Content "C:\Program Files (x86)\ossec-agent\ossec.log" -Tail 20
2026/09/19 21:10:43 sca: INFO: Loaded policy 'C:\Program Files (x86)\ossec-agent\ruleset\sca\cis_win11_enterprise.yml'
2026/09/19 21:10:43 sca: INFO: Starting Security Configuration Assessment scan.
2026/09/19 21:10:43 wazuh-agent: INFO: (6000): Starting daemon...
2026/09/19 21:10:43 wazuh-agent: INFO: (6010): File integrity monitoring scan frequency: 43200 seconds
2026/09/19 21:10:43 wazuh-agent: INFO: (6008): File integrity monitoring scan started.
2026/09/19 21:10:43 sca: INFO: Starting evaluation of policy: 'C:\Program Files (x86)\ossec-agent\ruleset\sca\cis_win11_enterprise.y
l'
2026/09/19 21:10:44 wazuh-agent: INFO: (4102): Connected to the server ([192.168.56.110]:1514/tcp).
2026/09/19 21:10:44 wazuh-modulesd:syscollector: INFO: Module started.
2026/09/19 21:10:44 wazuh-modulesd:syscollector: INFO: Starting evaluation.
2026/09/19 21:10:44 wazuh-modulesd:osquery: INFO: Module disabled. Exiting...
2026/09/19 21:10:44 rootcheck: INFO: Starting rootcheck scan.
2026/09/19 21:10:44 wazuh-agent: INFO: Started (pid: 19168).
2026/09/19 21:10:48 wazuh-agent: INFO: Agent is now online. Process unlocked, continuing...
2026/09/19 21:10:49 rootcheck: INFO: Ending rootcheck scan.
2026/09/19 21:10:52 wazuh-modulesd:syscollector: INFO: Evaluation finished.
2026/09/19 21:11:21 sca: INFO: Evaluation finished for policy 'C:\Program Files (x86)\ossec-agent\ruleset\sca\cis_win11_enterprise.y
l'
2026/09/19 21:11:21 sca: INFO: Security Configuration Assessment scan finished. Duration: 38 seconds.
2026/09/19 21:11:40 wazuh-agent: INFO: (6009): File integrity monitoring scan ended.
2026/09/19 21:11:40 wazuh-agent: INFO: FIM sync module started.
2026/09/19 21:11:43 wazuh-agent: INFO: (6012): Real-time file integrity monitoring started.
PS C:\Users\user>
```

≡

W.

Threat Hunting

API

default

a

?

25 hits

Sep 19, 2026 @ 21:58:20.240 - Sep 19, 2026 @ 22:13:20.240

Export Formatted

Reset view

746 available fields

Columns

Density

1 fields sorted

Full screen

Inspect document details	agent.name	rule.description	rule.level	rule.id
Sep 19, 2026 @ 22:11:08.9...	hostwin	File added to the system.	5	554

W.

Threat Hunting

API

default

a

25

Sep 19, 2026 @ 21:58:20.240

Export Formatted

Reset view

746 available fields

Columns

Density

timestamp	agent.name	rule.description
Sep 19, 2026 @ 22:11:08.9...	hostwin	File added to the system.
Sep 19, 2026 @ 22:11:00.4...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:11:00.4...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:11:00.4...	ubuntu	Successful
Sep 19, 2026 @ 22:11:00.4...	ubuntu	Successful
Sep 19, 2026 @ 22:09:26.3...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:09:26.3...	ubuntu	Successful
Sep 19, 2026 @ 22:09:26.3...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:09:26.3...	ubuntu	Successful
Sep 19, 2026 @ 22:09:26.3...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:09:26.3...	ubuntu	PAM: Log

Document Details

View surrounding documents

View single document

Table

JSON

t_index	wazuh-alerts-4.x-2026.09.19
t_agent.id	002
t_agent.ip	192.168.56.1
t_agent.name	hostwin
t_decoder.name	syscheck_new_entry
t_full_log	File 'c:\soc-test\alert-test.txt' added Mode: realtime
t_id	1789837868.1578164
t_input.type	log
t_location	syscheck
t_manager.name	ubuntu
t_rule.description	File added to the system.
# rule.firedtimes	2



W.

Threat Hunting

API

default

a



25		
Sep 19, 2026 @ 21:58:20.240		
Export Formatted Reset view 746 available fields Columns Density		
timestamp	agent.name	rule.desc
Sep 19, 2026 @ 22:11:08.9...	hostwin	File added
Sep 19, 2026 @ 22:11:00.4...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:11:00.4...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:11:00.4...	ubuntu	Successf
Sep 19, 2026 @ 22:11:00.4...	ubuntu	Successf
Sep 19, 2026 @ 22:09:26.3...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:09:26.3...	ubuntu	Successf
Sep 19, 2026 @ 22:09:26.3...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:09:26.3...	ubuntu	Successf
Sep 19, 2026 @ 22:09:26.3...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:09:26.3...	ubuntu	PAM: Log
Sep 19, 2026 @ 22:08:26.3...	ubuntu	Systemd

Document Details

View surrounding documents

View single document

rule.gpg13	4.11
rule.groups	ossec, syscheck, syscheck_entry_added, syscheck_file
rule.hipaa	164.312.c.1, 164.312.c.2
rule.id	554
rule.level	5
rule.mail	false
rule.nist_800_53	SI.7
rule.pci_dss	11.5
rule.tsc	PI1.4, PI1.5, CC6.1, CC6.8, CC7.2, CC7.3
syscheck.attrs_after	ARCHIVE
syscheck.event	added
syscheck.md5_after	9fd47f3a4ffc85f9ec23881fd2421f67
syscheck.mode	realtime
syscheck.mtime_after	Sep 19, 2026 @ 22:11:05.000
syscheck.path	c:\soc-test\alert-test.txt

4.7 Day 7 — Suspicious User Creation

The seventh stage of the project focused on monitoring user-account activity within the laboratory environment. The objective was to understand how the creation of a new user account can be detected and investigated as a potentially suspicious security event.

4.7.1 Test Scenario

A controlled user-account creation activity was performed within the laboratory environment.

The purpose of the test was to determine whether Wazuh could detect the creation of a new user account and provide relevant information for subsequent investigation.

The activity was performed only within the controlled laboratory environment for defensive security monitoring and analysis.

Testing Activity:

- Activity: Controlled user-account creation
- Monitoring Tool: Wazuh
- Purpose: Detect and investigate account-related security activity
- Observed Rule IDs: 60110, 60109, 60160

4.7.2 Detection

Wazuh was used to monitor the resulting system activity and identify the user-account creation event.

The generated security event was reviewed through the Wazuh Dashboard. Relevant information included the affected system, event description, timestamp, user-account details, and alert severity.

4.7.3 Investigation

The alert was investigated to determine:

- Name of the newly created account
- Account-creation timestamp
- System on which the account was created
- User responsible for the activity, where available
- Wazuh rule ID
- Alert severity
- Relevant event details

This information helps a SOC analyst determine whether the account creation was expected or requires further investigation.

4.7.4 Analysis

Unexpected account creation can be a security-relevant event because newly created accounts may potentially be used to obtain unauthorized access or maintain persistence.

In this project, the account creation was intentionally generated as part of a controlled laboratory test. Therefore, the activity was used to validate the monitoring and investigation process rather than represent an actual compromise.

4.7.5 MITRE ATT&CK Mapping

Where applicable, the observed user-account creation activity can be mapped to the relevant MITRE ATT&CK technique based on the exact event and system context.

The mapping should be recorded using the technique identified in the Wazuh alert or investigation evidence rather than assuming a technique without supporting evidence.

4.7.6 Response

Because the activity was intentionally generated in the laboratory, no production remediation was required.

The response process consisted of:

1. Validating the Wazuh alert.
2. Reviewing the account-creation event.
3. Identifying the affected system and account.
4. Recording relevant evidence.
5. Documenting the investigation findings.

In a real SOC environment, an unexpected account creation would require validation against authorized administrative activity and, where necessary, additional investigation.

4.7.7 Evidence

Evidence for this scenario may include:

- Wazuh Dashboard alert
- Account-creation event
- Newly created username
- Rule ID
- Alert severity

- Timestamp
- Relevant system information

4.7.8 Day 7 Outcome

The user-account monitoring scenario provided practical experience in detecting and investigating account-related security events. The exercise demonstrated how security monitoring can help analysts identify potentially unauthorized account activity and collect relevant evidence for further investigation.

```
PS C:\WINDOWS\system32> New-LocalUser -Name "SOC-TestUser" -Password (Read-Host -AsSecureString "Enter Password") -Description "Day 7 SOC user creation test"
Enter Password: *****

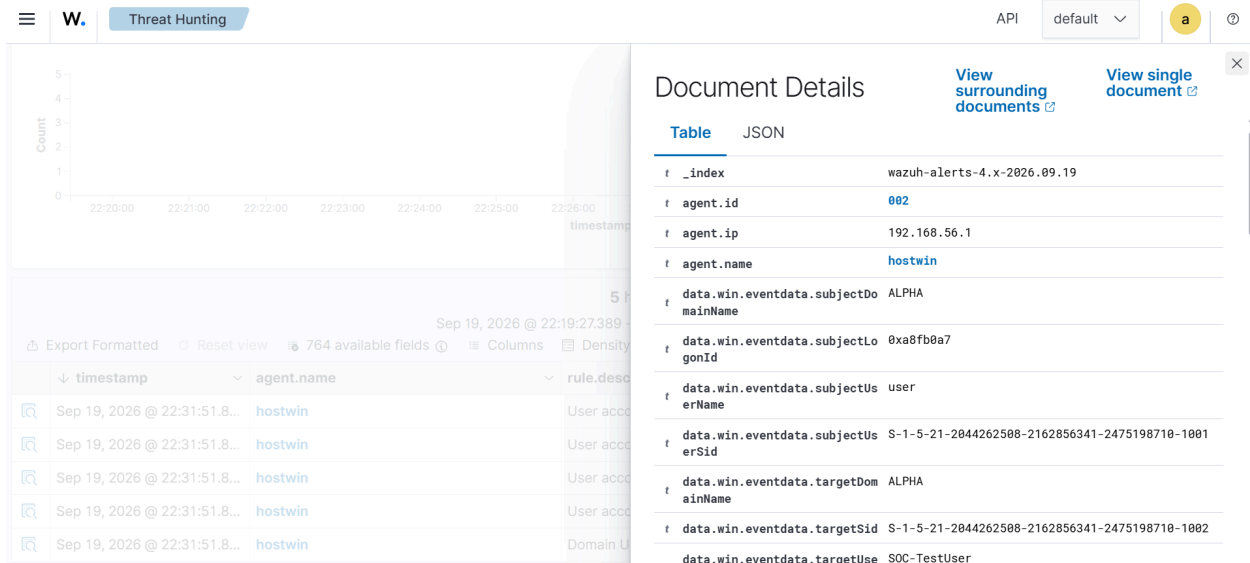
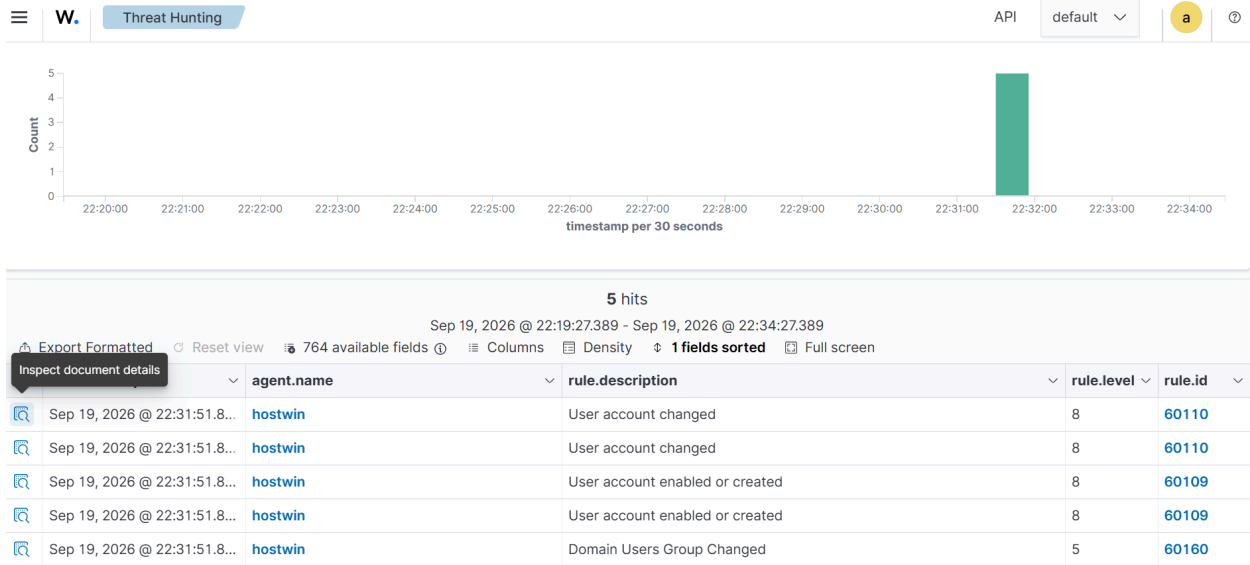
Name           Enabled Description
----
SOC-TestUser   True      Day 7 SOC user creation test
```

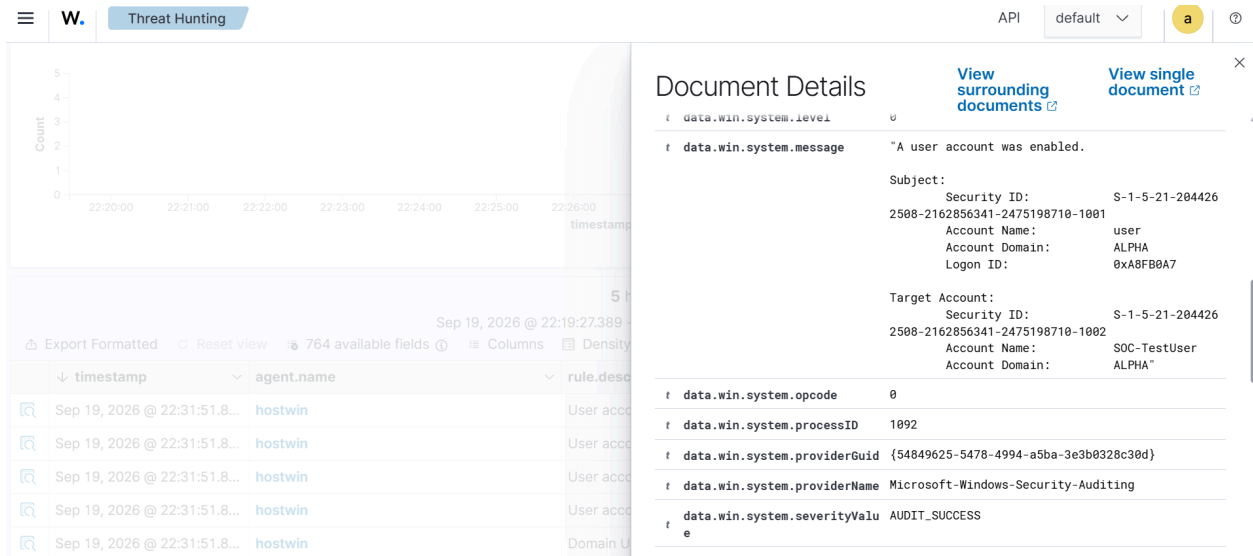
```
TimeCreated : 19/09/2026 22:31:47
Id           : 4720
ProviderName : Microsoft-Windows-Security-Auditing
Message      : A user account was created.

Subject:
  Security ID:      S-1-5-21-2044262508-2162856341-2475198710-1001
  Account Name:     user
  Account Domain:   ALPHA
  Logon ID:         0xA8FB0A7

New Account:
  Security ID:      S-1-5-21-2044262508-2162856341-2475198710-1002
  Account Name:     SOC-TestUser
  Account Domain:   ALPHA

Attributes:
  SAM Account Name:  SOC-TestUser
  Display Name:      <value not set>
  User Principal Name: -
  Home Directory:    <value not set>
  Home Drive:        <value not set>
  Script Path:       <value not set>
  Profile Path:      <value not set>
  User Workstations: <value not set>
  Password Last Set: <never>
  Account Expires:   <never>
  Primary Group ID:  513
  Allowed To Delegate To: -
  Old UAC Value:     0x0
  New UAC Value:     0x15
  User Account Control:
    Account Disabled
    'Password Not Required' - Enabled
```





4.8 Day 8 — Suricata IDS Monitoring

The eighth stage of the project focused on using Suricata as a network intrusion detection system (IDS) on the Windows host machine. The objective was to monitor network activity and identify security-related events generated by Suricata.

4.8.1 Test Environment

Suricata was configured and used on the Windows 11 host machine as part of the network security monitoring setup.

The host machine was also configured with the Wazuh Agent:

- **Operating System:** Windows 11 Pro
- **Wazuh Agent:** hostwin

- **Agent ID:** 002
- **Wazuh Manager:** 192.168.56.110

The Ubuntu virtual machine continued to function as the Wazuh Manager and Dashboard environment, but Suricata testing for this stage was performed on the Windows host.

4.8.2 Suricata Monitoring

Suricata was used to monitor network activity on the host machine and identify potentially suspicious network events.

The purpose of the configuration was to provide network-level security monitoring alongside the endpoint monitoring already provided by Wazuh.

4.8.3 Detection

During the testing, a Suricata-related security event was detected and processed through the monitoring environment.

The observed Wazuh alert included:

- **Rule ID:** 86601
- **Detection Source:** Suricata
- **Monitored System:** Windows host
- **Wazuh Agent:** hostwin
- **Agent ID:** 002

The alert provided evidence that network security monitoring was able to identify the tested activity.

4.8.4 Investigation

The generated alert was reviewed through the Wazuh Dashboard.

The investigation focused on:

- Rule ID
- Alert timestamp
- Source information
- Destination information
- Network activity details
- Alert severity
- Suricata event information
- Affected Wazuh agent

The available event information was reviewed to understand the nature and context of the detected network activity.

4.8.5 Analysis

Network-based detection provides an additional layer of visibility beyond endpoint monitoring. Suricata can identify suspicious network activity and generate security events that can be investigated by a SOC analyst.

In this laboratory scenario, the Suricata event was generated as part of controlled security testing. The alert was therefore analyzed as a laboratory detection rather than treated as evidence of a real-world compromise.

4.8.6 Response

The activity was performed within the controlled laboratory environment, so no production remediation was required.

The response process consisted of:

1. Validating the Suricata alert.
2. Reviewing Rule 86601.
3. Examining the available network event information.
4. Correlating the event with the monitored host.
5. Documenting the detection and investigation findings.

In a real SOC environment, the source and destination information, affected systems, and nature of the network activity would be investigated further before determining an appropriate response.

4.8.7 Evidence

The main evidence collected during this stage included:

- Wazuh Dashboard alert
- Suricata-related Rule ID 86601
- Windows host information
- Wazuh Agent hostwin
- Network event details
- Alert timestamp and severity

4.8.8 Day 8 Outcome

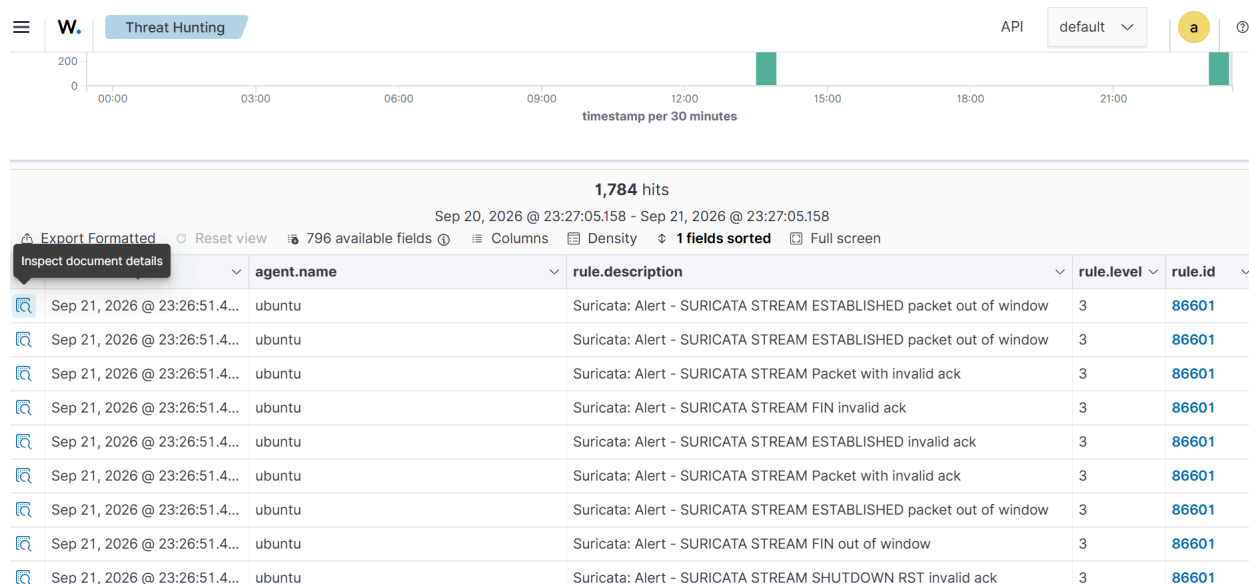
The Suricata testing stage provided practical experience with network intrusion detection on a Windows host environment. The generated security event was reviewed through Wazuh,

demonstrating how network-based detection can provide additional security visibility alongside endpoint monitoring.

This stage strengthened practical skills in IDS monitoring, alert investigation, network-event analysis, and SOC documentation.

```
PS C:\WINDOWS\system32> Test-NetConnection 192.168.56.110 -Port 22

ComputerName      : 192.168.56.110
RemoteAddress     : 192.168.56.110
RemotePort        : 22
InterfaceAlias    : Ethernet 2
SourceAddress     : 192.168.56.1
TcpTestSucceeded  : True
```



1,784

Sep 20, 2026 @ 23:27:05.158

Export Formatted

Reset view

796 available fields

Columns

Density

timestamp	agent.name	rule.desc
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:43.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:43.4...	ubuntu	Suricata:...

Document Details

[View surrounding documents](#)

[View single document](#)

TableJSON

_index	wazuh-alerts-4.x-2026.09.21
agent.id	000
agent.name	ubuntu
data.alert.action	allowed
data.alert.category	Generic Protocol Command Decode
data.alert.gid	1
data.alert.rev	2
data.alert.severity	3
data.alert.signature	SURICATA STREAM ESTABLISHED packet out of window
data.alert.signature_id	2210020
data.app_proto	tls
data.dest_ip	192.168.56.1
data.dest_port	57965

1,784

Sep 20, 2026 @ 23:27:05.158

Export Formatted

Reset view

796 available fields

Columns

Density

timestamp	agent.name	rule.desc
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:43.4...	ubuntu	Suricata:...
Sep 21, 2026 @ 23:26:43.4...	ubuntu	Suricata:...

Document Details

[View surrounding documents](#)

[View single document](#)

TableJSON

data.direction	to_client
data.event_type	alert
data.flow.bytes_toclient	1176608
data.flow.bytes_toserver	62621
data.flow.dest_ip	192.168.56.110
data.flow.dest_port	443
data.flow.pkts_toclient	835
data.flow.pkts_toserver	443
data.flow.src_ip	192.168.56.1
data.flow.src_port	57965
data.flow.start	2026-09-21T18:26:35.585792+0000
data.flow_id	1108584093411473.000000
data.in_iface	enp8s8
data.pkt_src	wire/pcap
data.proto	TCP

W.

Threat Hunting

API default a

1,784

Sep 20, 2026 @ 23:27:05,158

Export Formatted Reset view 796 available fields Columns Density

timestamp	agent.name	rule.description
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:51.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:43.4...	ubuntu	Suricata: ...
Sep 21, 2026 @ 23:26:43.4...	ubuntu	Suricata: ...

Document Details

View surrounding documents View single document

data.tls.ja3s.string	771,4866,43-51
data.tls.version	TLS 1.3
decoder.name	json
id	1790015211.2966426
input.type	log
location	/var/log/suricata/eve.json
manager.name	ubuntu
rule.description	Suricata: Alert - SURICATA STREAM ESTABLISHED packet out of window
rule.firedtimes	806
rule.groups	ids, suricata
rule.id	86601
rule.level	3
rule.mail	false
timestamp	Sep 21, 2026 @ 23:26:51.481

4.9 Day 9 — MITRE ATT&CK Mapping & Incident Investigation

The ninth stage of the project focused on analyzing the security events generated during the laboratory exercises and converting the collected alerts and logs into structured incident investigations.

The objective was to understand how a SOC analyst reviews security events, identifies relevant evidence, maps applicable activities to the MITRE ATT&CK framework, and documents investigation findings.

4.9.1 Security Event Investigation

During this stage, previously generated Wazuh alerts and Windows security events were reviewed in greater detail.

The investigation focused on identifying:

- Event and rule IDs
- Event timestamps
- Affected systems
- User or account information
- Source information
- Logon and authentication details
- Alert severity
- Event descriptions
- Available evidence

The collected information was used to understand the context of each security event rather than analyzing individual alerts in isolation.

4.9.2 Windows Event ID 4625 Analysis

Windows Security Event ID **4625** was analyzed as part of the authentication-event investigation.

Event ID 4625 represents a failed Windows logon attempt. The event details were reviewed to identify information such as the account involved, logon type, failure reason, timestamp, and available source information.

The event was treated as an important authentication-related security event. Multiple failed logon events can provide additional context when investigating potentially suspicious authentication activity.

The Event ID 4625 record was therefore included as supporting evidence in the incident investigation process.

4.9.3 MITRE ATT&CK Mapping

The observed security activities were reviewed against the MITRE ATT&CK framework to identify applicable adversary techniques.

For the SSH brute-force scenario, the Wazuh alert was mapped to:

- **Tactic:** Credential Access
- **Technique:** T1110 – Brute Force

The mapping was based on the observed repeated SSH authentication failures and the corresponding Wazuh alert.

Other security events were reviewed for applicable MITRE ATT&CK techniques where sufficient evidence was available.

4.9.4 Incident Investigation Process

The project investigations followed a structured SOC workflow:

Detect → Validate → Investigate → Analyze → Respond → Document

During the investigation stage, alerts were validated against the available logs and event information. Relevant evidence was then reviewed to understand what occurred, which system was affected, and what information could be established from the available data.

4.9.5 Incident Documentation

The security scenarios were organized into structured incident reports containing:

- Incident description
- Detection source
- Event/rule ID
- Timestamp
- Source and target information
- Affected user or system
- Evidence
- Investigation findings
- Analysis
- MITRE ATT&CK mapping where applicable
- Response
- Recommendations

The following activities were included in the project investigation:

Security Activity	Evidence / Tool
SSH Brute Force	Wazuh Rule 5712 , MITRE T1110
Network Scanning	Nmap results — 22/tcp, 443/tcp
File Integrity Monitoring	Wazuh Rule 554
User-Account Activity	Wazuh Rules 60109, 60110, 60160
Network Security Monitoring	Suricata/Wazuh Rule 86601
Failed Windows Logon	Windows Event ID 4625

4.9.6 Evidence Correlation

Where relevant, multiple pieces of evidence were reviewed together to provide better context for an investigation.

For example, Windows Event ID 4625 provided authentication-related information, while Wazuh alerts provided centralized monitoring and alert information. Combining available endpoint and security-monitoring evidence can help an analyst understand the sequence and context of suspicious activity.

4.9.7 Day 9 Outcome

Day 9 provided practical experience in security-event investigation, Windows event analysis, MITRE ATT&CK mapping, evidence correlation, and incident documentation.

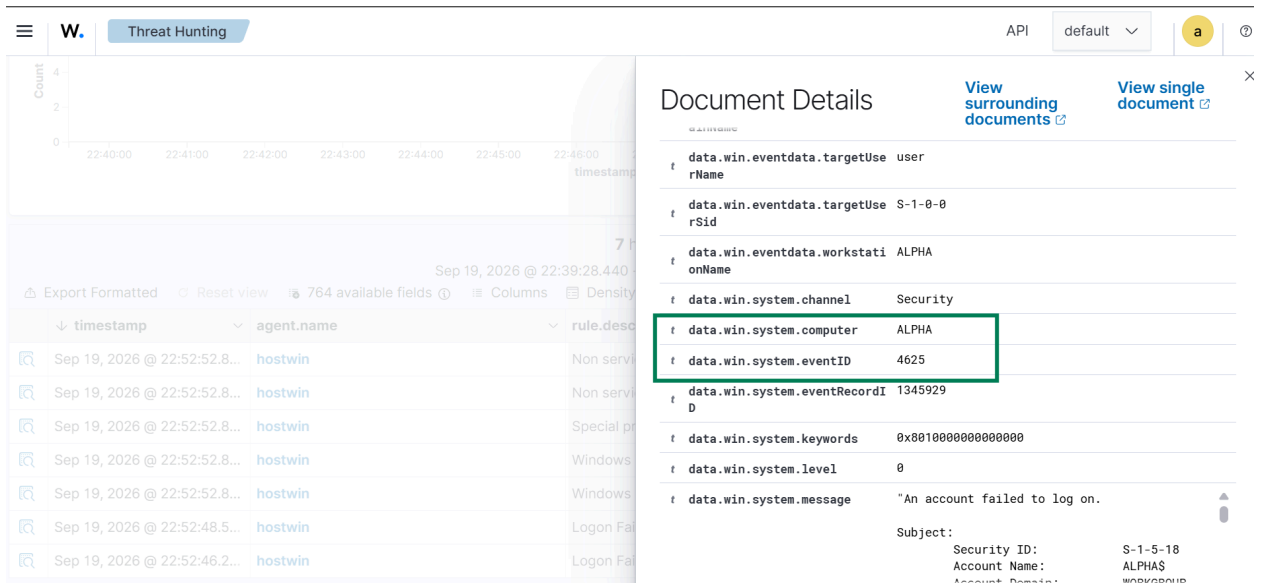
The collected alerts and logs were transformed into structured investigation records, demonstrating an end-to-end SOC workflow from security-event detection through analysis and documentation.

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 5
```

```
ProviderName: Microsoft-Windows-Security-Auditing
```

TimeCreated	Id	LevelDisplayName	Message
19/09/2026 22:52:44	4625	Information	An account failed to log on....
19/09/2026 22:52:41	4625	Information	An account failed to log on....
19/09/2026 12:38:37	4625	Information	An account failed to log on....
18/09/2026 12:15:18	4625	Information	An account failed to log on....
17/09/2026 15:28:47	4625	Information	An account failed to log on....

```
PS C:\WINDOWS\system32>
```



4.10 Day 10 — Documentation & Finalization

The final stage of the SOC Monitoring & Incident Detection Lab focused on reviewing, organizing, and documenting the work completed throughout the project. The objective was to convert the practical lab activities, security alerts, investigation findings, and evidence into a structured cybersecurity project suitable for technical documentation and portfolio presentation.

4.10.1 Project Review

The complete laboratory environment and previously performed security scenarios were reviewed to ensure that the documented information accurately represented the activities performed during the project.

The review covered:

- Wazuh Manager and Dashboard configuration
- Windows endpoint monitoring through the Wazuh Agent
- SSH brute-force detection
- Network scanning using Nmap
- File Integrity Monitoring
- User-account activity monitoring
- Suricata-based network security monitoring
- Windows Event ID 4625 analysis
- MITRE ATT&CK mapping
- Incident investigation and documentation

4.10.2 Evidence Organization

Screenshots and other available evidence were reviewed and organized according to the relevant security scenario. Evidence included Wazuh alerts, terminal outputs, security events, rule IDs, timestamps, and other relevant investigation details.

Only available and relevant evidence was included in the final documentation. Deleted or unavailable screenshots were not used as project evidence.

4.10.3 Architecture Documentation

The final SOC laboratory architecture was prepared to clearly represent the relationship between the Windows host, Wazuh Agent, Suricata, Ubuntu Wazuh server, Wazuh Dashboard, and incident investigation process.

The architecture diagram was designed to show the flow of security events from monitored systems to centralized monitoring and subsequent investigation.

4.10.4 Incident Report Preparation

The security scenarios investigated during the project were organized into structured incident reports. Each report included relevant information such as:

- Incident description
- Detection source
- Rule or event ID
- Timestamp
- Source and target information
- Affected system or user
- Evidence
- Investigation findings
- Analysis
- MITRE ATT&CK mapping where applicable
- Response actions
- Recommendations

4.10.5 Final Project Documentation

The project documentation was reviewed and structured into a professional report covering the laboratory environment, tools, configuration, security scenarios, investigations, MITRE ATT&CK mapping, findings, challenges, and results.

The documentation was also prepared as the basis for creating a GitHub repository and project README containing the main technical details and selected evidence.

4.10.6 Final Verification

A final review was performed to check:

- Accuracy of technical information
- Consistency of IP addresses and system names
- Correct rule and event IDs
- Availability of supporting evidence
- Correct placement of security tools in the architecture
- Consistency between screenshots and written descriptions
- Appropriate MITRE ATT&CK mappings
- Removal of unsupported or unavailable evidence

4.10.7 Project Outcome

The final stage completed the documentation and organization of the SOC Monitoring & Incident Detection Lab. The project now provides a structured record of practical activities involving

security monitoring, alert detection, log analysis, incident investigation, network security monitoring, endpoint monitoring, and MITRE ATT&CK analysis.

The completed documentation can be used as a technical portfolio project to demonstrate practical Blue Team and SOC-related skills.

5. Security Monitoring & Incident Detection

The SOC Monitoring & Incident Detection Lab was designed to provide centralized security monitoring and detection of controlled security activities. Wazuh was used as the primary security monitoring platform, while the Wazuh Agent provided endpoint visibility on the Windows host. Suricata was used to provide additional network security monitoring.

5.1 Wazuh Security Monitoring

Wazuh was used to collect, process, and display security-related events through the Wazuh Manager and Dashboard. The Ubuntu virtual machine hosted the Wazuh Manager and Dashboard, while the Windows host was monitored using the Wazuh Agent.

The Wazuh Dashboard provided a centralized interface for reviewing security alerts, event details, rule IDs, timestamps, severity levels, and available source or target information.

5.2 Security Alerts & Detection

During the laboratory activities, different security events were generated and analyzed under controlled conditions. These included SSH authentication failures, network scanning, file-system changes, user-account activity, Suricata network security events, and Windows authentication failures.

The monitoring process allowed security events to be identified and reviewed using available logs and alerts. Relevant evidence was examined to understand the nature of each event and determine whether further investigation was required.

The main detection activities performed during the project included:

- SSH brute-force detection using Wazuh.
- Network reconnaissance using Nmap.
- File Integrity Monitoring using Wazuh.
- User-account activity monitoring.
- Network security monitoring using Suricata.
- Analysis of Windows Event ID 4625 for failed logon activity.

The monitoring process followed a basic SOC workflow:

Detect → Validate → Investigate → Analyze → Respond → Document

This workflow helped organize the investigation of security events and provided practical experience in security monitoring and alert analysis.

6. Incident Investigation & MITRE ATT&CK

Incident investigation was an important part of the SOC Monitoring & Incident Detection Lab. After security events were detected, relevant alerts and logs were reviewed to understand the activity, identify available evidence, and determine the nature of the event.

6.1 Incident Investigation Process

The investigation process followed a structured SOC workflow:

Detect → Validate → Investigate → Analyze → Respond → Document

During an investigation, the available alert information was reviewed, including:

- Timestamp
- Event or rule ID
- Alert severity
- Source information
- Target or affected system
- Username or account information, where available
- Event description
- Network or system activity
- Related security evidence

The available information was then analyzed to determine whether the activity represented normal, expected, or potentially suspicious behavior within the laboratory environment.

6.2 Incident Analysis

Several controlled security scenarios were investigated during the project. These included SSH authentication failures, network scanning, file-system changes, user-account activity, Suricata network security events, and Windows failed-logon events.

For example, the SSH brute-force scenario generated Wazuh alerts associated with repeated authentication failures. The investigation included reviewing the relevant Wazuh rule, source information, target user, timestamps, and available MITRE ATT&CK information.

Windows Event ID 4625 was also examined as supporting evidence for failed Windows logon activity. The event was reviewed for relevant authentication information such as the account, timestamp, logon information, and available source details.

All investigations were performed within the controlled laboratory environment for educational and defensive security purposes.

6.3 MITRE ATT&CK Mapping

The MITRE ATT&CK framework was used to provide a standardized way of describing relevant attacker behavior observed during the laboratory activities.

The SSH brute-force scenario was associated with:

- **Tactic:** Credential Access
- **Technique:** T1110 — Brute Force
- **Activity:** Repeated SSH authentication failures

The mapping was based on the observed security activity and the MITRE ATT&CK information associated with the Wazuh detection.

Other laboratory events were analyzed individually, and MITRE ATT&CK techniques were considered only where the available evidence supported the mapping. This helped avoid assigning techniques without sufficient evidence.

Using MITRE ATT&CK during the investigation improved understanding of how individual security events can be connected to a broader attack-behavior framework and helped make the incident documentation more structured.

7. Challenges & Troubleshooting

During the development and testing of the SOC Monitoring & Incident Detection Lab, several configuration and resource-related challenges were encountered. Troubleshooting these issues provided practical experience in configuring security monitoring components and diagnosing problems in a virtualized laboratory environment.

7.1 Challenges Faced

The main challenges encountered during the project included:

- Wazuh configuration and service-related issues during the initial setup.
- Wazuh Agent configuration and communication with the Wazuh Manager.
- File Integrity Monitoring configuration and alert verification.
- Suricata monitoring and integration with the security monitoring environment.
- Limited system resources while running multiple virtualized environments.
- Reviewing and validating security alerts to ensure that the available evidence matched the documented activity.

7.2 Troubleshooting

Wazuh configuration files and services were reviewed when configuration or service issues occurred. Configuration errors were corrected and the Wazuh services were restarted and verified before continuing with the monitoring activities.

For the Windows Wazuh Agent, the agent configuration was reviewed to ensure that the correct Wazuh Manager address was specified and that the agent service was running. The agent was then verified through the Wazuh environment.

File Integrity Monitoring required additional verification because file changes did not always immediately produce the expected alert. The monitoring configuration and generated events were reviewed to determine whether the activity was being correctly detected.

Suricata monitoring was also reviewed as part of the network security monitoring setup. The generated Suricata-related alert was verified through Wazuh and analyzed using the available event information.

The laboratory was also affected by limited hardware resources. Running multiple virtual machines simultaneously caused performance limitations on the host system. Resource allocation was therefore adjusted where possible to maintain a usable laboratory environment.

These troubleshooting activities helped improve understanding of security-tool configuration, agent communication, event generation, alert verification, and resource management in a practical SOC laboratory.

8. Results & Skills Demonstrated

The SOC Monitoring & Incident Detection Lab provided practical experience in building a small-scale security monitoring environment and investigating controlled security events. The project demonstrated the use of security monitoring, endpoint monitoring, network security monitoring, log analysis, and incident investigation techniques.

8.1 Project Results & Findings

The project successfully established a functional laboratory environment using Windows, Ubuntu, VirtualBox, Wazuh, and Suricata.

The following activities were successfully performed and analyzed:

- Wazuh Manager and Dashboard were configured on the Ubuntu virtual machine.
- A Wazuh Agent was configured on the Windows host for endpoint monitoring.
- Controlled SSH authentication failures were detected using Wazuh.
- Network scanning was performed against the Ubuntu server using Nmap.
- File-system activity was monitored using Wazuh File Integrity Monitoring.
- User-account activity was monitored through security events.
- Suricata-generated network security activity was detected and reviewed through Wazuh.
- Windows Event ID 4625 was analyzed as a failed logon event.
- Relevant security events were investigated using available evidence.
- MITRE ATT&CK was used to map applicable security activity, including T1110 for the observed brute-force scenario.
- Security findings and evidence were organized into structured incident documentation.

The project demonstrated that centralized security monitoring can provide visibility into different types of endpoint, authentication, file-system, and network-related security events within a controlled environment.

8.2 Skills Demonstrated

The project provided practical experience in the following areas:

- **SIEM & Security Monitoring:** Wazuh Manager, Wazuh Dashboard, alerts, and event monitoring.
- **Log Analysis:** Reviewing timestamps, rule IDs, event details, severity, and available source information.
- **Incident Investigation:** Validating alerts, collecting evidence, analyzing events, and documenting findings.
- **Network Security Monitoring:** Using Nmap for network reconnaissance and Suricata for network intrusion detection.

- **Endpoint Monitoring:** Monitoring Windows activity through the Wazuh Agent.
- **File Integrity Monitoring:** Reviewing file-system changes and related security alerts.
- **MITRE ATT&CK:** Mapping applicable security activity to relevant techniques.
- **Incident Documentation:** Preparing structured investigation records and organizing supporting evidence.
- **Troubleshooting:** Diagnosing configuration, connectivity, monitoring, and resource-related issues.

Overall, the project provided practical exposure to the workflow used in basic SOC and Blue Team activities, from security-event detection through investigation and documentation.

9. Conclusion

9.1 Project Summary

The SOC Monitoring & Incident Detection Lab was developed as a practical cybersecurity laboratory to gain hands-on experience in security monitoring, alert detection, log analysis, and incident investigation.

The project used Windows 11, Ubuntu, VirtualBox, Wazuh, Suricata, and Nmap to create a controlled environment for generating and analyzing security events. Different scenarios, including SSH authentication failures, network scanning, file-system activity, user-account activity, network security events, and Windows failed-logon events, were investigated using available security evidence.

The project also incorporated MITRE ATT&CK mapping and structured incident documentation to provide a more organized approach to security-event analysis.

9.2 Key Learning Outcomes

Through this project, practical experience was gained in:

- Configuring and using Wazuh for security monitoring.
- Monitoring an endpoint using the Wazuh Agent.
- Reviewing and analyzing security alerts and logs.
- Investigating authentication and system-related events.
- Performing controlled network reconnaissance using Nmap.
- Using Suricata for network security monitoring.
- Applying File Integrity Monitoring concepts.
- Analyzing Windows Event ID 4625.
- Mapping applicable security activity to MITRE ATT&CK.
- Documenting security incidents and investigation findings.
- Troubleshooting security-tool configuration and connectivity issues.

9.3 Future Improvements

The laboratory can be expanded in the future by adding additional endpoints, more security monitoring tools, automated response capabilities, and additional attack simulations.

Future improvements could also include integrating additional log sources, developing custom Wazuh detection rules, creating more detailed incident response playbooks, and expanding the laboratory to simulate a larger SOC environment.

Overall, the project provided practical experience in the fundamental processes of security monitoring and Blue Team operations and established a foundation for further development of SOC and cybersecurity skills.

10. References

1. Wazuh. *Wazuh Documentation*. Wazuh Documentation.
2. Suricata. *Suricata Documentation*. Open Information Security Foundation (OISF).
3. MITRE ATT&CK. *MITRE ATT&CK Framework*. MITRE.
4. Nmap. *Nmap Reference Guide*. Nmap Project.
5. Microsoft. *Windows Security Auditing and Event Documentation*. Microsoft Learn.
6. Oracle. *Oracle VM VirtualBox Documentation*. Oracle.

Appendices

Appendix A — Architecture Diagram

Final architecture diagram of the SOC Monitoring & Incident Detection Lab, showing the Windows host, Wazuh Agent, Suricata, Ubuntu Wazuh Manager, Wazuh Dashboard, and investigation workflow.

Appendix B — Selected Wazuh Alerts

Selected screenshots of relevant Wazuh alerts generated during the laboratory activities, including SSH brute-force detection, File Integrity Monitoring, user-account activity, and Suricata-related detection.

Appendix C — Nmap Scan Evidence

Screenshot showing the Nmap scan performed from Kali Linux against the Ubuntu server, including the identified open ports.

Appendix D — Suricata Evidence

Screenshot showing the Suricata-related security alert detected through Wazuh on the Windows host.

Appendix E — Windows Event Evidence

Screenshot showing Windows Security Event ID 4625 and the relevant event information used during the investigation.

Appendix F — Incident Reports

Structured incident reports prepared for the security scenarios investigated during the project.